NTT / FFT 多项式板子

本板子默认 C++17，所有整数统一使用 `ll = long long`，多项式统一使用 `vector<ll>`，并约定 `a[i]` 是 x^i 的系数。代码中的短变量名方便赛场抄写；代码块只保留接口、参数和关键前置条件，完整说明写在代码块外。

0. 怎么选

需求	选择	结果	主要限制
模 `998244353` 卷积、形式幂级数	NTT / FPS 主板子	完全精确	变换长度不超过 2^{23}
普通整数卷积	单次打包 FFT	浮点计算后取整	必须自行确认精度
任意模数卷积	三模 NTT + CRT	满足系数界时完全精确	长度不超过 2^{21}
多项式逆、ln、exp、sqrt、pow	FPS 主板子	模 `998244353`	各操作有常数项条件
少量互异点只求一个值	独立拉格朗日	完全精确	普通点 `O(n^2)`，连续点 `O(n)`
多点求值、插值	乘积树扩展	完全精确	插值点必须互异
多项式复合、复合逆	进阶 FPS 扩展	完全精确	常数较大，适合中等长度
已知若干项求远处项	BM + Bostan-Mori	完全精确	数列满足短线性递推
卷积型递推 DP	CDQ 卷积	完全精确	转移核只与下标差有关
XOR / OR / AND 卷积	FWT	完全精确	长度补成 2 的幂

若题目模数就是 `998244353`，优先使用 NTT。若要求任意模数且不能接受精度风险，优先使用三模 NTT。FFT 常数小，但它不是整数精确算法，数据越长、系数越大，舍入风险越高。

---

1. 高性能 NTT 与形式幂级数全家桶

下面整块代码可以直接复制。它包含：

- NTT、卷积、加减、截断；
- 求导、积分、逆、对数、指数；
- Tonelli-Shanks 模平方根、多项式平方根；
- 多项式非负整数幂；
- 多项式除法与取模；

- 泰勒平移 $F(x+c)$。

NTT 使用缓存的位逆序数组和单位根，同一长度下重复做多项式运算时无需反复求根。短多项式自动切换到朴素乘法，通常比强行 NTT 更快。

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using poly = vector<ll>;

const ll mod = 998244353, G = 3;

    ll norm(ll x) { x %= mod; return x < 0 ? x + mod : x; }
    ll qp(ll a, ll b) { // a^b mod mod; b>=0
        ll r = 1;
        for (a = norm(a); b; b >>= 1, a = a * a % mod)
            if (b & 1) r = r * a % mod;
        return r;
    }

    vector<int> rv;
    poly rt{0, 1};

    void ntt(poly &a) { // a.size() 必须为 2 的幂；原地正变换
        int n = a.size(), k = __builtin_ctz(n);
        if ((int)rv.size() != n) {
            rv.assign(n, 0);
            for (int i = 1; i < n; ++i)
                rv[i] = (rv[i >> 1] >> 1) | ((i & 1) << (k - 1));
        }
        if ((int)rt.size() < n) {
            int k = __builtin_ctz((int)rt.size());
            rt.resize(n);
            while ((1 << k) < n) {
                ll e = qp(G, (mod - 1) >> (k + 1));
                for (int i = 1 << (k - 1); i < (1 << k); ++i) {
                    rt[i << 1] = rt[i];
                    rt[i << 1 | 1] = rt[i] * e % mod;
                }
                ++k;
            }
        }
        for (int i = 0; i < n; ++i) if (i < rv[i]) swap(a[i], a[rv[i]]);
        for (int m = 1; m < n; m <<= 1)
            for (int p = 0; p < n; p += m << 1)
                for (int i = 0; i < m; ++i) {
                    ll x = a[p + i], y = a[p + i + m] * rt[m + i] % mod;
                    a[p + i] = x + y < mod ? x + y : x + y - mod;
                    a[p + i + m] = x - y >= 0 ? x - y : x - y + mod;
                }
    }
```

```
    void intt(poly &a) { // a.size() 必须为 2 的幂；原地逆变换
        reverse(a.begin() + 1, a.end());
        ntt(a);
        ll iv = qp(a.size(), mod - 2);
        for (ll &x : a) x = x * iv % mod;
    }

    void trim(poly &a) { // 删除高位 0；零多项式变成空 vector
        while (!a.empty() && norm(a.back()) == 0) a.pop_back();
    }

    poly pre(poly a, int n) { // 返回 a mod x^n，恰好 n 项
        a.resize(n);
        for (ll &x : a) x = norm(x);
        return a;
    }

    poly add(const poly &a, const poly &b) { // 返回 a+b
        poly c(max(a.size(), b.size()));
        for (int i = 0; i < (int)c.size(); ++i) {
            ll x = i < (int)a.size() ? norm(a[i]) : 0;
            ll y = i < (int)b.size() ? norm(b[i]) : 0;
            c[i] = x + y < mod ? x + y : x + y - mod;
        }
        trim(c); return c;
    }

    poly sub(const poly &a, const poly &b) { // 返回 a-b
        poly c(max(a.size(), b.size()));
        for (int i = 0; i < (int)c.size(); ++i) {
            ll x = i < (int)a.size() ? norm(a[i]) : 0;
            ll y = i < (int)b.size() ? norm(b[i]) : 0;
            c[i] = x - y >= 0 ? x - y : x - y + mod;
        }
        trim(c); return c;
    }

    poly mul(poly a, poly b) { // 返回卷积；a、b 可含负系数
        if (a.empty() || b.empty()) return {};
        int z = a.size() + b.size() - 1;
        for (ll &x : a) x = norm(x);
        for (ll &x : b) x = norm(x);
        if (min(a.size(), b.size()) <= 32) {
            poly c(z);
            for (int i = 0; i < (int)a.size(); ++i)
                for (int j = 0; j < (int)b.size(); ++j)
                    c[i + j] = (c[i + j] + a[i] * b[j]) % mod;
            return c;
        }
        int n = 1; while (n < z) n <<= 1;
        assert(n <= (1 << 23));
        a.resize(n); b.resize(n); ntt(a); ntt(b);
```

```
    for (int i = 0; i < n; ++i) a[i] = a[i] * b[i] % mod;
    intt(a); a.resize(z); return a;
}

poly der(const poly &a) { // 返回 A'(x)
    if (a.size() <= 1) return {};
    poly b(a.size() - 1);
    for (int i = 1; i < (int)a.size(); ++i) b[i - 1] = norm(a[i]) * i % mod;
    return b;
}

poly inte(const poly &a) { // 返回积分，常数项取 0；要求 a.size()<mod
    static poly iv{0, 1};
    int n = a.size();
    if ((int)iv.size() <= n) {
        int s = iv.size(); iv.resize(n + 1);
        for (int i = max(2, s); i <= n; ++i)
            iv[i] = (mod - mod / i) * iv[mod % i] % mod;
    }
    poly b(n + 1);
    for (int i = 0; i < n; ++i) b[i + 1] = norm(a[i]) * iv[i + 1] % mod;
    return b;
}

poly inv(const poly &a, int n) { // 返回 A^{-1} mod x^n；要求 n>=1 且 a[0]!=0
    assert(n >= 1 && !a.empty() && norm(a[0]) != 0);
    poly b{qp(a[0], mod - 2)};
    for (int m = 2; m < 2 * n; m <<= 1) {
        int z = min(m, n);
        poly t = mul(pre(a, z), b); t.resize(z);
        for (ll &x : t) x = x ? mod - x : 0;
        t[0] = (t[0] + 2) % mod;
        b = mul(b, t); b.resize(z);
    }
    b.resize(n); return b;
}

poly ln(const poly &a, int n) { // 返回 ln(A) mod x^n；要求 n>=1 且 a[0]=1
    assert(n >= 1 && !a.empty() && norm(a[0]) == 1);
    poly b = mul(der(pre(a, n)), inv(a, n));
    b.resize(max(0, n - 1));
    b = inte(b); b.resize(n); return b;
}

poly exp(const poly &a, int n) { // 返回 exp(A) mod x^n；要求 n>=1 且 a[0]=0
    assert(n >= 1 && (a.empty() || norm(a[0]) == 0));
    poly b{1};
    for (int m = 2; m < 2 * n; m <<= 1) {
        int z = min(m, n);
        poly t = ln(b, z); t.resize(z);
        for (int i = 0; i < z; ++i) {
            ll x = i < (int)a.size() ? norm(a[i]) : 0;
```

```
            t[i] = x - t[i] >= 0 ? x - t[i] : x - t[i] + mod;
        }
        t[0] = (t[0] + 1) % mod;
        b = mul(b, t); b.resize(z);
    }
    b.resize(n); return b;
}

ll msqrt(ll a) { // 返回 a 的一个模平方根；无解返回 -1（mod 必须为奇质数）
    a = norm(a);
    if (a == 0) return 0;
    if (qp(a, (mod - 1) / 2) != 1) return -1;
    if (mod % 4 == 3) return qp(a, (mod + 1) / 4);
    ll q = mod - 1, s = 0;
    while (!(q & 1)) q >>= 1, ++s;
    ll z = 2; while (qp(z, (mod - 1) / 2) == 1) ++z;
    ll c = qp(z, q), x = qp(a, (q + 1) / 2), t = qp(a, q), m = s;
    while (t != 1) {
        ll i = 1, y = t * t % mod;
        while (y != 1) y = y * y % mod, ++i;
        ll b = qp(c, 1LL << (m - i - 1));
        x = x * b % mod; c = b * b % mod; t = t * c % mod; m = i;
    }
    return x;
}

poly sqrt_nz(const poly &a, int n, ll r) { // a[0]!=0，r^2=a[0]；返回平方根前 n 项
    poly b{r}; const ll i2 = (mod + 1) / 2;
    for (int m = 2; m < 2 * n; m <<= 1) {
        int z = min(m, n);
        poly t = mul(pre(a, z), inv(b, z)); t.resize(z);
        b.resize(z);
        for (int i = 0; i < z; ++i) b[i] = (b[i] + t[i]) * i2 % mod;
    }
    b.resize(n); return b;
}

poly sqrt(const poly &a, int n) { // 返回 B，使 B^2=A mod x^n；无解返回空 vector
    assert(n >= 1);
    int p = 0;
    while (p < n && (p >= (int)a.size() || norm(a[p]) == 0)) ++p;
    if (p == n) return poly(n);
    if (p & 1) return {};
    ll r = msqrt(a[p]); if (r == -1) return {};
    poly c(a.begin() + p, a.end());
    poly b = sqrt_nz(c, n - p / 2, r), ans(p / 2);
    ans.insert(ans.end(), b.begin(), b.end()); ans.resize(n);
    return ans;
}

poly pow(const poly &a, ll k, int n) { // 返回 A^k mod x^n；要求 k>=0、n>=1
    assert(k >= 0 && n >= 1);
```

```
        poly ans(n);
        if (k == 0) { ans[0] = 1; return ans; }
        int p = 0;
        while (p < n && (p >= (int)a.size() || norm(a[p]) == 0)) ++p;
        if (p == n || (p && k > (n - 1) / p)) return ans;
        int sh = p * k, m = n - sh;
        ll x = norm(a[p]), ix = qp(x, mod - 2);
        poly c(m);
        for (int i = 0; i < m && i + p < (int)a.size(); ++i)
            c[i] = norm(a[i + p]) * ix % mod;
        poly t = ln(c, m);
        for (ll &v : t) v = v * (k % mod) % mod;
        poly b = exp(t, m); ll s = qp(x, k);
        for (int i = 0; i < m; ++i) ans[i + sh] = b[i] * s % mod;
        return ans;
    }

    pair<poly, poly> divmod(poly a, poly b) { // 返回 {商,余数}；除数 b 非零
        for (ll &x : a) x = norm(x);
        for (ll &x : b) x = norm(x);
        trim(a); trim(b); assert(!b.empty());
        if (a.size() < b.size()) return {{}, a};
        int n = a.size(), m = b.size(), k = n - m + 1;
        poly ra = a, rb = b; reverse(ra.begin(), ra.end()); reverse(rb.begin(),
rb.end());
        poly q = mul(pre(ra, k), inv(pre(rb, k), k)); q.resize(k);
        reverse(q.begin(), q.end());
        poly r = sub(a, mul(b, q)); r.resize(m - 1); trim(q); trim(r);
        return {q, r};
    }

    poly shift(const poly &a, ll c) { // 返回 A(x+c)；要求 a.size()<mod
        int n = a.size(); if (!n) return {};
        c = norm(c);
        poly fac(n, 1), ifac(n, 1), x(n, 1), y(n);
        for (int i = 1; i < n; ++i) fac[i] = fac[i - 1] * i % mod;
        ifac[n - 1] = qp(fac[n - 1], mod - 2);
        for (int i = n - 1; i; --i) ifac[i - 1] = ifac[i] * i % mod;
        for (int i = 1; i < n; ++i) x[i] = x[i - 1] * c % mod;
        for (int i = 0; i < n; ++i) y[n - 1 - i] = norm(a[i]) * fac[i] % mod;
        for (int i = 0; i < n; ++i) x[i] = x[i] * ifac[i] % mod;
        poly z = mul(x, y), b(n);
        for (int i = 0; i < n; ++i) b[i] = z[n - 1 - i] * ifac[i] % mod;
        return b;
    }
```

1.1 基础接口与传参

复制主代码后可直接调用：

```
poly a = {1, 2, 3};        // A(x)=1+2x+3x^2
poly b = {4, 5};           // B(x)=4+5x
poly c = mul(a, b);        // c={4,13,22,15}

poly f = pre(a, 5);        // A mod x^5，返回恰好 5 项
poly g = add(a, b);        // A+B
poly h = sub(a, b);        // A-B，系数已规范到 [0,mod)
```

`mul(a,b)` 接受空多项式，也允许输入负系数，返回长度恰好为 `a.size()+b.size()-1`。卷积复杂度为 $O(n \log n)$；较短一侧长度不超过 32 时自动使用朴素乘法。

`pre(a,n)` 与多数 FPS 接口都按"项数"而不是"次数"传参。传 `n=5` 表示只保留 $x^0 \sim x^4$。

1.2 求导、积分、逆、对数、指数

```
poly d = der(a);           // A'(x)
poly s = inte(a);          // 积分，常数项固定为 0
poly iv = inv(a, n);       // A^{-1} mod x^n；a[0]!=0
poly l = ln(a, n);         // ln(A) mod x^n；a[0]==1
poly e = exp(a, n);        // exp(A) mod x^n；a[0]==0
```

- `inv(a,n)` 求 B 使 $AB \equiv 1 \pmod{x^n}$，常数项只需非零。
- `ln(a,n)` 使用 $\ln A = \int A'/A \, dx$，形式幂级数中要求 `a[0]==1`。
- `exp(a,n)` 求唯一满足 $\ln B = A$ 且 $B(0) = 1$ 的多项式，要求 `a[0]==0`。
- `inte` 需要求 $1, 2, \ldots$ 的逆元，因此实际使用时要求项数小于模数；竞赛数据一般天然满足。

这些操作均返回恰好 `n` 项，复杂度为 $O(n \log n)$，但 `exp`、`sqrt` 的常数明显大于一次卷积。

1.3 平方根与幂

```
ll r = msqrt(x);           // r*r%mod==x；不存在返回 -1
poly s = sqrt(a, n);       // S^2=A mod x^n；不存在返回空 vector
poly p = pow(a, k, n);     // A^k mod x^n；k 为非负 ll
```

`sqrt(a,n)` 支持开头有若干个零的多项式。第一个非零项下标必须为偶数，且该项系数必须是模 `998244353` 的二次剩余；否则返回空 `vector`。非零平方根通常有两个，代码返回 Tonelli-Shanks 找到的其中一个，另一个是它的相反数。

`pow(a,k,n)` 支持 `a[0]==0`，也支持很大的 `k`。当最低非零项为 $a_p x^p$ 时，结果最低项次数为 pk；若它已经不小于 `n`，直接返回全零。`k=0` 按形式幂级数约定返回 1，包括 0^0 的模板约定。

1.4 除法、取模与泰勒平移

```
auto [q, r] = divmod(a, b); // A=BQ+R，deg(R)<deg(B)；b 非零
poly t = shift(a, c);       // 返回 A(x+c)
```

`divmod` 会自动删除输入的高位零，零多项式用空 `vector` 表示。多项式取模可写成 `divmod(a,b).second`。

`shift(a,c)` 计算泰勒平移，输出长度与 `a` 相同，复杂度 $O(n \log n)$。例如 `a={1,2,3}`、`c=1` 得到 $A(x+1)=6+8x+3x^2$，即 `{6,8,3}`。

多项式 gcd 和模多项式快速幂可直接建立在 `divmod` 上：

```
    poly pgcd(poly a, poly b) { // 返回首一 gcd(A,B)
        for (ll &x : a) x = norm(x); for (ll &x : b) x = norm(x);
        trim(a); trim(b);
        while (!b.empty()) { poly r = divmod(a, b).second; a.swap(b); b.swap(r); }
        if (!a.empty()) {
            ll z = qp(a.back(), mod - 2);
            for (ll &x : a) x = x * z % mod;
        }
        return a;
    }

    poly powmod(poly a, ll k, const poly &m) { // 返回 A^k mod M；k>=0，M 非零
        assert(k >= 0); poly mm = m; trim(mm); assert(!mm.empty());
        a = divmod(a, mm).second;
        poly r = divmod(poly{1}, mm).second;
        for (; k; k >>= 1, a = divmod(mul(a, a), mm).second)
            if (k & 1) r = divmod(mul(r, a), mm).second;
        return r;
    }
```

```
poly g = pgcd(a, b);          // 首项系数（最高次项）规范为 1
poly r = powmod(a, k, m);     // A^k mod M(x)
```

`pgcd(0,0)` 返回空 `vector`。`powmod` 常用于多项式判定、有限域构造和模特征多项式计算；每一步包含乘法和多项式取模，复杂度约为 $O(M(n)\log n \log k)$，其中 `n=deg(M)`。

1.5 三角函数、反三角函数与 Bostan-Mori

本段继续依赖第 1 节的 FPS 主代码。形式幂级数的 `sin/cos/asin/atan` 通常要求常数项为 0。

```
    pair<poly, poly> sincos(const poly &a, int n) { // 返回 {sin(A),cos(A)} mod x^n；a[0]=0
        assert(n >= 1 && (a.empty() || norm(a[0]) == 0));
        ll I = qp(G, (mod - 1) / 4), i2 = (mod + 1) / 2;
        poly z = pre(a, n);
        for (ll &x : z) x = x * I % mod;
```

```
        poly e = exp(z, n), ie = inv(e, n), s(n), c(n);
        ll ii = qp(I, mod - 2) * i2 % mod;
        for (int i = 0; i < n; ++i) {
            s[i] = (e[i] - ie[i] + mod) % mod * ii % mod;
            c[i] = (e[i] + ie[i]) % mod * i2 % mod;
        }
        return {s, c};
    }

    poly tan(const poly &a, int n) { // 返回 tan(A) mod x^n；a[0]=0
        auto [s, c] = sincos(a, n);
        poly t = mul(s, inv(c, n)); t.resize(n); return t;
    }

    poly asin(const poly &a, int n) { // 返回 asin(A) mod x^n；a[0]=0
        assert(n >= 1 && (a.empty() || norm(a[0]) == 0));
        poly z = pre(mul(pre(a, n), pre(a, n)), n);
        for (ll &x : z) x = x ? mod - x : 0;
        z[0] = (z[0] + 1) % mod;
        poly s = sqrt(z, n); assert(!s.empty());
        if (s[0] != 1) for (ll &x : s) x = x ? mod - x : 0;
        poly b = mul(der(pre(a, n)), inv(s, n));
        b.resize(n - 1); b = inte(b); b.resize(n); return b;
    }

    poly atan(const poly &a, int n) { // 返回 atan(A) mod x^n；a[0]=0
        assert(n >= 1 && (a.empty() || norm(a[0]) == 0));
        poly z = pre(mul(pre(a, n), pre(a, n)), n);
        z[0] = (z[0] + 1) % mod;
        poly b = mul(der(pre(a, n)), inv(z, n));
        b.resize(n - 1); b = inte(b); b.resize(n); return b;
    }

    ll bostan(poly p, poly q, ll k) { // 返回 [x^k] P(x)/Q(x)；k>=0，q[0]!=0
        assert(k >= 0 && !q.empty() && norm(q[0]) != 0);
        for (ll &x : p) x = norm(x);
        for (ll &x : q) x = norm(x);
        while (k) {
            poly nq = q;
            for (int i = 1; i < (int)nq.size(); i += 2) nq[i] = nq[i] ? mod - nq[i]
: 0;
            poly a = mul(p, nq), b = mul(q, nq), np, qq;
            for (int i = k & 1; i < (int)a.size(); i += 2) np.push_back(a[i]);
            for (int i = 0; i < (int)b.size(); i += 2) qq.push_back(b[i]);
            p.swap(np); q.swap(qq); k >>= 1;
        }
        return (p.empty() ? 0 : p[0]) * qp(q[0], mod - 2) % mod;
    }

    ll linear_k(const poly &a, const poly &c, ll k) { // a: 前 d 项；c[i]: a[n-i-1]
的系数
        int d = a.size(); assert(d > 0 && (int)c.size() == d && k >= 0);
```

```
        poly q(d + 1); q[0] = 1;
        for (int i = 0; i < d; ++i) q[i + 1] = norm(-c[i]);
        poly p = mul(a, q); p.resize(d);
        return bostan(p, q, k);
    }
```

调用方式：

```
auto [s, c] = sincos(a, n); // sin(A)、cos(A) 的前 n 项
poly t = tan(a, n);         // tan(A)
poly u = asin(a, n);        // asin(A)
poly v = atan(a, n);        // atan(A)

ll x = bostan(p, q, k);     // [x^k] P(x)/Q(x)
ll y = linear_k(a, c, k);   // d 阶线性递推的第 k 项（从 0 开始）
```

`linear_k` 中 `a={a0,...,a(d-1)}` 是初值，`c={c0,...,c(d-1)}` 表示

$$a_n = c_0 a_{n-1} + c_1 a_{n-2} + \cdots + c_{d-1} a_{n-d}.$$

例如 Fibonacci 可传 `a={0,1}`、`c={1,1}`。Bostan-Mori 的复杂度为 $O(M(d)\log k)$，其中 $M(d)$ 是长度 d 的卷积复杂度；适合阶数较大且 `k` 可达 `1e18` 的递推。若阶数很小，Kitamasa 的 $O(d^2 \log k)$ 可能常数更小。

1.6 BM、多项式复合、复合逆与 k 次根

本段依赖第 1 节的 FPS 主代码。`bm` 从数列前若干项求最短线性递推，返回格式可直接传给 `linear_k`。通常至少给出最短递推阶数两倍左右的项，才有足够信息唯一确定递推。

`compose(f,g,n)` 计算 $F(G(x)) \bmod x^n$；这里要求 `g[0]=0`，因此 `f` 的前 `n` 项足够。实现采用分块预处理 $1, G, \ldots, G^B$，代码短但常数和内存都较大，适合中等长度，不应代替一次普通卷积。

```
    poly bm(const poly &s) { // 返回递推 c：s[n]=sum(c[i-1]*s[n-i])
        poly C{1}, B{1};
        ll b = 1; int L = 0, m = 1;
        for (int n = 0; n < (int)s.size(); ++n) {
            ll d = norm(s[n]);
            for (int i = 1; i <= L; ++i) d = (d + C[i] * norm(s[n - i])) % mod;
            if (!d) { ++m; continue; }
            poly T = C; ll z = d * qp(b, mod - 2) % mod;
            if ((int)C.size() < (int)B.size() + m) C.resize(B.size() + m);
            for (int i = 0; i < (int)B.size(); ++i)
                C[i + m] = (C[i + m] - z * B[i] % mod + mod) % mod;
            if (2 * L <= n) { L = n + 1 - L; B = T; b = d; m = 1; }
            else ++m;
        }
        C.erase(C.begin()); C.resize(L);
        for (ll &x : C) x = x ? mod - x : 0;
        return C;
    }
```

```
    poly compose(const poly &f, const poly &g, int n) { // F(G(x)) mod x^n；n>=1，
g[0]=0
        assert(n >= 1 && (g.empty() || norm(g[0]) == 0));
        int B = (int)sqrtl(n) + 1, z = min(n, (int)f.size());
        poly gg = pre(g, n); trim(gg);
        vector<poly> pw(B + 1); pw[0] = {1};
        for (int i = 1; i <= B; ++i) {
            pw[i] = mul(pw[i - 1], gg);
            if ((int)pw[i].size() > n) pw[i].resize(n);
            trim(pw[i]);
        }
        poly gb = pw[B], cur{1}, ans(n);
        for (int l = 0; l < z; l += B) {
            poly h(n);
            for (int j = 0; j < B && l + j < z; ++j) {
                ll v = norm(f[l + j]); if (!v) continue;
                for (int k = 0; k < (int)pw[j].size(); ++k)
                    h[k] = (h[k] + v * pw[j][k]) % mod;
            }
            trim(h);
            if (!h.empty() && !cur.empty()) {
                poly v = mul(cur, h); if ((int)v.size() > n) v.resize(n);
                for (int i = 0; i < (int)v.size(); ++i) ans[i] = (ans[i] + v[i]) %
mod;
            }
            if (l + B < z) {
                cur = mul(cur, gb); if ((int)cur.size() > n) cur.resize(n);
                trim(cur);
            }
        }
        return ans;
    }

    poly revert(const poly &f, int n) { // 返回 G：F(G(x))=x mod x^n；f[0]=0，f[1]!=0
        assert(n >= 1 && f.size() >= 2 && norm(f[0]) == 0 && norm(f[1]) != 0);
        if (n == 1) return {0};
        poly g{0, qp(f[1], mod - 2)};
        for (int m = 2; m < n; m <<= 1) {
            int z = min(2 * m, n);
            poly fg = compose(pre(f, z), g, z);
            fg[1] = (fg[1] - 1 + mod) % mod;
            poly dg = compose(der(pre(f, z)), g, z);
            poly d = mul(fg, inv(dg, z)); d.resize(z); g.resize(z);
            for (int i = 0; i < z; ++i) g[i] = (g[i] - d[i] + mod) % mod;
        }
        g.resize(n); return g;
    }

    poly root1(const poly &a, ll k, int n) { // 返回常数项为 1 的 k 次根；a[0]=1，
k%mod!=0
        assert(k >= 1 && k % mod != 0 && n >= 1 && !a.empty() && norm(a[0]) == 1);
        poly b = ln(a, n); ll ik = qp(k % mod, mod - 2);
```

```
        for (ll &x : b) x = x * ik % mod;
        return exp(b, n);
    }
```

调用方式：

```
poly c = bm(seq);              // 从 seq 推出最短递推系数
ll x = linear_k(init, c, k);  // 已知该递推的前 c.size() 项，求第 k 项
poly h = compose(f, g, n);     // F(G(x)) mod x^n；g[0]=0
poly g = revert(f, n);         // F(G(x))=x mod x^n
poly r = root1(a, k, n);       // r^k=a，且 r[0]=1
```

`bm` 的复杂度为 $O(m^2)$，`m` 是提供的数列项数。全零序列会返回空递推，此时后续任意项直接为 0，不要把空初值传给 `linear_k`。`compose` 的时间复杂度约为 $O(\sqrt{n}\,M(n))$、内存约为 $O(n\sqrt{n})$；`revert` 在倍增过程中重复调用复合，量级相同但常数更大。长度上万时应寻找更专业的快速复合实现，不要直接使用这份赛场短板子。

`root1` 只处理常数项为 1 的标准情形，并要求 `k` 在模意义下可逆。一般多项式 k 次根还要处理最低非零项次数能否被 `k` 整除、常数项是否存在 k 次剩余，不能只套 `ln/exp`。

---

2. 拉格朗日插值、多点求值与快速插值

2.1 只求一个点：独立拉格朗日模块

若只给少量互异点 (x_i, y_i)，并且只求一个 $F(k)$，不需要抄 NTT。`lag(x,y,k)` 用普通拉格朗日插值，时间 $O(n^2)$、额外空间 $O(1)$；要求 `x.size()==y.size()`，所有 `x[i]` 在模 `P` 下互异，且 `P` 是质数。

若已知连续点 `F(0),F(1),...,F(n-1)`，调用 `lag0(y,k)`，时间和空间都是 $O(n)$。它利用

$$\prod_{j\neq i}(i-j) = i!(-1)^{n-1-i}(n-1-i)!$$

把每项分母变成阶乘。两个函数都只依赖同一代码块里的三个短辅助函数，可以单独抄。

```
using ll = long long;
const ll P = 998244353;

ll lg_norm(ll x) { x %= P; return x < 0 ? x + P : x; }
ll lg_pw(ll a, ll b) { // a：底数；b：指数；返回 a^b mod P
    ll r = 1;
    for (a = lg_norm(a); b; b >>= 1, a = a * a % P)
        if (b & 1) r = r * a % P;
    return r;
}

ll lag(const vector<ll> &x, const vector<ll> &y, ll k) {
    // x：横坐标；y：对应函数值；k：待求值横坐标
    int n = x.size();
    k = lg_norm(k);
```

```
    for (int i = 0; i < n; i++) if (lg_norm(x[i]) == k) return lg_norm(y[i]);
    ll r = 0;
    for (int i = 0; i < n; i++) {
        ll a = 1, b = 1, xi = lg_norm(x[i]);
        for (int j = 0; j < n; j++) if (i != j) {
            a = a * lg_norm(k - x[j]) % P;
            b = b * lg_norm(xi - x[j]) % P;
        }
        r = (r + lg_norm(y[i]) * a % P * lg_pw(b, P - 2)) % P;
    }
    return r;
}

ll lag0(const vector<ll> &y, ll k) {
    // y[i]: F(i), 0<=i<n; k: 待求值横坐标
    int n = y.size();
    k = lg_norm(k);
    if (k < n) return lg_norm(y[k]);
    vector<ll> pr(n + 1, 1), sf(n + 1, 1), fac(n, 1), iv(n, 1);
    for (int i = 0; i < n; i++) pr[i + 1] = pr[i] * lg_norm(k - i) % P;
    for (int i = n - 1; i >= 0; i--) sf[i] = sf[i + 1] * lg_norm(k - i) % P;
    for (int i = 1; i < n; i++) fac[i] = fac[i - 1] * i % P;
    if (n) {
        iv[n - 1] = lg_pw(fac[n - 1], P - 2);
        for (int i = n - 1; i; i--) iv[i - 1] = iv[i] * i % P;
    }
    ll r = 0;
    for (int i = 0; i < n; i++) {
        ll v = lg_norm(y[i]) * pr[i] % P * sf[i + 1] % P;
        v = v * iv[i] % P * iv[n - 1 - i] % P;
        if ((n - 1 - i) & 1) v = v ? P - v : 0;
        r += v; if (r >= P) r -= P;
    }
    return r;
}
```

`lag0` 要求 `n<P`。传入的 `k` 会先模 `P`；如果 `k mod P` 正好落在 `0..n-1`，直接返回已知值。普通 `lag` 适合 `n` 较小的单次询问；需要还原全部系数或处理 10^5 级别互异点时，使用下一节。

2.2 多点求值与快速插值

本段依赖第 1 节的完整 FPS 主代码。乘积树维护

$$T_{l,r}(x)=\prod_{i=l}^{r-1}(x-x_i).$$

多点求值沿树递归取模；插值用

$$F(x)=\sum_i \frac{y_i}{T'(x_i)}\frac{T(x)}{x-x_i}.$$

```
    struct Tree {
        int n;
        poly x;
```

```
    vector<poly> tr;

    Tree(const poly &x): n(x.size()), x(x), tr(max(1, 4 * (int)x.size())) {
        if (n) build(1, 0, n);
    }

    void build(int p, int l, int r) {
        if (r - l == 1) { tr[p] = {norm(-x[l]), 1}; return; }
        int m = (l + r) >> 1;
        build(p << 1, l, m); build(p << 1 | 1, m, r);
        tr[p] = mul(tr[p << 1], tr[p << 1 | 1]);
    }

    poly rem(poly f, const poly &g) { // 返回 f mod g，g 非零
        trim(f);
        if (f.size() < g.size()) return f;
        return divmod(f, g).second;
    }

    void get(int p, int l, int r, poly f, poly &ans) {
        f = rem(f, tr[p]);
        if (r - l == 1) { ans[l] = f.empty() ? 0 : f[0]; return; }
        int m = (l + r) >> 1;
        get(p << 1, l, m, f, ans); get(p << 1 | 1, m, r, f, ans);
    }

    poly eval(const poly &f) { // 返回 {f(x[0]),...,f(x[n-1])}
        poly ans(n); if (n) get(1, 0, n, f, ans); return ans;
    }

    poly calc(int p, int l, int r, const poly &v) {
        if (r - l == 1) return {v[l]};
        int m = (l + r) >> 1;
        poly a = calc(p << 1, l, m, v), b = calc(p << 1 | 1, m, r, v);
        return add(mul(a, tr[p << 1 | 1]), mul(b, tr[p << 1]));
    }

    poly interp(const poly &y) { // y.size()==x.size()，x 两两不同
        assert((int)y.size() == n);
        if (!n) return {};
        poly w = eval(der(tr[1])), v(n);
        for (int i = 0; i < n; ++i) {
            assert(w[i] != 0);
            v[i] = norm(y[i]) * qp(w[i], mod - 2) % mod;
        }
        poly f = calc(1, 0, n, v); f.resize(n); return f;
    }
};

ll eval0(const poly &y, ll x) { // y[i]=F(i)，deg(F)<y.size()，返回 F(x)
    int n = y.size(); assert(n > 0 && n < mod); x = norm(x);
    if (x < n) return norm(y[x]);
```

```
        poly pr(n + 1, 1), sf(n + 1, 1), fac(n, 1), ifac(n, 1);
        for (int i = 0; i < n; ++i) pr[i + 1] = pr[i] * norm(x - i) % mod;
        for (int i = n - 1; i >= 0; --i) sf[i] = sf[i + 1] * norm(x - i) % mod;
        for (int i = 1; i < n; ++i) fac[i] = fac[i - 1] * i % mod;
        ifac[n - 1] = qp(fac[n - 1], mod - 2);
        for (int i = n - 1; i; --i) ifac[i - 1] = ifac[i] * i % mod;
        ll ans = 0;
        for (int i = 0; i < n; ++i) {
            ll v = norm(y[i]) * pr[i] % mod * sf[i + 1] % mod;
            v = v * ifac[i] % mod * ifac[n - 1 - i] % mod;
            if ((n - 1 - i) & 1) v = v ? mod - v : 0;
            ans += v; if (ans >= mod) ans -= mod;
        }
        return ans;
    }
```

调用方式：

```
poly xs = {0, 1, 2};
Tree t(xs);                 // xs：所有求值点
poly ys = t.eval(f);        // ys[i]=f(xs[i])
poly g = t.interp(ys);      // 恢复次数 < xs.size() 的多项式
ll v = eval0(ys, x);        // 仅当 ys[i]=F(i) 时，O(n) 求 F(x)
```

构造、求值、插值的复杂度均为 $O(n\log^2 n)$，内存为 $O(n\log n)$。同一组 `xs` 上对多个多项式求值时复用同一个 `Tree`，可省去重复建树。插值要求所有 `xs[i]` 在模意义下两两不同，否则 $T'(x_i)=0$，问题无唯一解。

当已知的是连续点 `F(0),F(1),...,F(n-1)`，只询问一个 `F(x)` 时用 `eval0`，时间 $O(n)$、内存 $O(n)$，不必建立乘积树。若有很多查询，可先用多点求值，或针对连续点预处理阶乘后复用。

---

3. 循环卷积、CDQ 卷积递推与 FWT

这一节都是“按题复制”的扩展，依赖第 1 节 `FPS` 主板子。普通卷积的下标规则是 `i+j`；循环卷积把下标对 `n` 取模，负循环卷积则利用 $x^n=-1$ 折叠。

3.1 循环卷积与负循环卷积

```
    poly cyclic(const poly &a, const poly &b, int n) { // A*B mod (x^n-1)：a,b 长度
<=n
        assert(n >= 1 && (int)a.size() <= n && (int)b.size() <= n);
        poly c = mul(pre(a, n), pre(b, n)), r(n);
        for (int i = 0; i < (int)c.size(); ++i) r[i % n] = (r[i % n] + c[i]) % mod;
        return r;
    }

    poly negacyclic(const poly &a, const poly &b, int n) { // A*B mod (x^n+1)：a,b 长
度<=n
        assert(n >= 1 && (int)a.size() <= n && (int)b.size() <= n);
```

```
        poly c = mul(pre(a, n), pre(b, n)), r(n);
        for (int i = 0; i < (int)c.size(); ++i) {
            if (i < n) r[i] = (r[i] + c[i]) % mod;
            else r[i - n] = (r[i - n] - c[i] + mod) % mod;
        }
        return r;
    }
```

```
poly c = cyclic(a, b, n);     // 下标 i+j 按 n 取模
poly d = negacyclic(a, b, n); // 超过 n 的项折回时取负号
```

两个接口先做一次普通卷积再折叠，复杂度 $O(n \log n)$。若输入本身超过 `n` 项，应先按对应规则折叠到长度 `n`，不能直接截断。

3.2 CDQ 卷积递推

求解

$$f_i = h_i + \sum_{j=0}^{i-1} f_j g_{i-j}$$

时，可以先把 `f` 初始化为外加项 `h`，再用 CDQ 分治把已经求出的左半段贡献给右半段。

```
    void cdq_dfs(poly &f, const poly &g, int l, int r) {
        if (r - l == 1) return;
        int m = (l + r) >> 1; cdq_dfs(f, g, l, m);
        poly a(f.begin() + l, f.begin() + m);
        int z = min((int)g.size(), r - l);
        poly b(g.begin(), g.begin() + z), c = mul(a, b);
        for (int i = m; i < r && i - l < (int)c.size(); ++i) {
            f[i] += c[i - l]; if (f[i] >= mod) f[i] -= mod;
        }
        cdq_dfs(f, g, m, r);
    }

    void cdq(poly &f, const poly &g) { // f 初值为 h；原地求上述递推，g[0] 不参与
        for (ll &x : f) x = norm(x);
        if (!f.empty()) cdq_dfs(f, g, 0, f.size());
    }
```

```
poly f = h;               // h[i] 是不依赖之前状态的外加项
cdq(f, g);                // 得到全部 f[i]；使用 g[i-j]
```

复杂度为 $O(n \log^2 n)$。`g[0]` 不参与，因为求和只取 `j<i`。若转移还依赖位置、需要在叶子处执行额外操作，直接改 `cdq_dfs` 的叶子逻辑；若递推能直接写成生成函数 $F = H/(1-G)$，一次 FPS 求逆通常更快。

3.3 XOR、OR、AND 卷积

普通 NTT 处理下标相加，FWT 处理下标的位运算。`type=0/1/2` 分别表示 XOR、OR、AND。

```
    void fwt(poly &a, int type, bool iv) { // type: 0=XOR,1=OR,2=AND; iv=true 为逆变换
        int n = a.size(); assert(n && !(n & (n - 1)) && 0 <= type && type <= 2);
        const ll i2 = (mod + 1) / 2;
        for (int m = 1; m < n; m <<= 1)
            for (int p = 0; p < n; p += m << 1)
                for (int i = 0; i < m; ++i) {
                    ll x = a[p + i], y = a[p + i + m];
                    if (type == 0) {
                        a[p + i] = (x + y) % mod;
                        a[p + i + m] = (x - y + mod) % mod;
                        if (iv) a[p + i] = a[p + i] * i2 % mod,
                                a[p + i + m] = a[p + i + m] * i2 % mod;
                    } else if (type == 1) {
                        if (!iv) a[p + i + m] = (y + x) % mod;
                        else a[p + i + m] = (y - x + mod) % mod;
                    } else {
                        if (!iv) a[p + i] = (x + y) % mod;
                        else a[p + i] = (x - y + mod) % mod;
                    }
                }
    }

    poly fwt_conv(poly a, poly b, int type) { // c[k]=sum a[i]b[j]，其中 i op j=k
        if (a.empty() || b.empty()) return {};
        int n = 1; while (n < (int)max(a.size(), b.size())) n <<= 1;
        a.resize(n); b.resize(n);
        for (ll &x : a) x = norm(x); for (ll &x : b) x = norm(x);
        fwt(a, type, false); fwt(b, type, false);
        for (int i = 0; i < n; ++i) a[i] = a[i] * b[i] % mod;
        fwt(a, type, true); return a;
    }

    poly sub_conv(const poly &a, const poly &b) { // c[S]=sum_{T subset S}
a[T]b[S\T]
        assert(a.size() == b.size());
        int n = a.size(); assert(n && !(n & (n - 1)));
        int k = __builtin_ctz(n);
        vector<poly> A(k + 1, poly(n)), B(k + 1, poly(n)), C(k + 1, poly(n));
        for (int s = 0; s < n; ++s) {
            int c = __builtin_popcount(s);
            A[c][s] = norm(a[s]); B[c][s] = norm(b[s]);
        }
        for (int i = 0; i <= k; ++i)
            for (int b = 0; b < k; ++b)
                for (int s = 0; s < n; ++s) if (s >> b & 1) {
                    A[i][s] += A[i][s ^ (1 << b)]; if (A[i][s] >= mod) A[i][s] -=
mod;
```

```
                    B[i][s] += B[i][s ^ (1 << b)]; if (B[i][s] >= mod) B[i][s] -=
mod;
                }
        for (int c = 0; c <= k; ++c)
            for (int i = 0; i <= c; ++i)
                for (int s = 0; s < n; ++s)
                    C[c][s] = (C[c][s] + A[i][s] * B[c - i][s]) % mod;
        for (int i = 0; i <= k; ++i)
            for (int b = 0; b < k; ++b)
                for (int s = 0; s < n; ++s) if (s >> b & 1)
                    C[i][s] = (C[i][s] - C[i][s ^ (1 << b)] + mod) % mod;
        poly c(n);
        for (int s = 0; s < n; ++s) c[s] = C[__builtin_popcount(s)][s];
        return c;
    }
```

```
poly x = fwt_conv(a, b, 0); // x[k]=sum_{i xor j=k} a[i]b[j]
poly o = fwt_conv(a, b, 1); // o[k]=sum_{i or  j=k} a[i]b[j]
poly d = fwt_conv(a, b, 2); // d[k]=sum_{i and j=k} a[i]b[j]
poly s = sub_conv(a, b);    // s[S]=sum_{T subset S} a[T]b[S\T]
```

`fwt_conv` 会把长度补到不小于两输入长度的最小 2 的幂，返回该长度，三种 FWT 都是 $O(n \log n)$。`sub_conv` 要求两输入长度相同且为 2^k，使用“按集合大小分层 + 子集 zeta 变换”，复杂度 $O(k^2 2^k)$、内存 $O(k2^k)$。子集卷积不是简单的 OR 卷积，不要混用。

---

4. 单次打包 FFT：普通整数卷积

下面用一个复数数组同时装入两个实序列，只做一次正变换和一次逆变换。与“两个数组分别 FFT”相比少一次正变换，适合不取模的整数卷积。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

    const double PI = acos(-1.0);

    struct C {
        double x, y;
        C(double x = 0, double y = 0): x(x), y(y) {}
        C operator + (const C &b) const { return {x + b.x, y + b.y}; }
        C operator - (const C &b) const { return {x - b.x, y - b.y}; }
        C operator * (const C &b) const { return {x * b.x - y * b.y, x * b.y + y *
b.x}; }
        C conj() const { return {x, -y}; }
    };

    void fft(vector<C> &a, bool iv) { // iv=false 正变换，iv=true 逆变换
        int n = a.size();
        for (int i = 1, j = 0; i < n; ++i) {
```

```
            int b = n >> 1;
            for (; j & b; b >>= 1) j ^= b;
            j ^= b; if (i < j) swap(a[i], a[j]);
        }
        for (int m = 2; m <= n; m <<= 1) {
            double z = 2 * PI / m * (iv ? -1 : 1);
            C wm(cos(z), sin(z));
            for (int p = 0; p < n; p += m) {
                C w(1, 0);
                for (int i = 0; i < (m >> 1); ++i, w = w * wm) {
                    C x = a[p + i], y = a[p + i + (m >> 1)] * w;
                    a[p + i] = x + y; a[p + i + (m >> 1)] = x - y;
                }
            }
        }
        if (iv) for (C &x : a) x.x /= n, x.y /= n;
    }

    vector<ll> fft_mul(const vector<ll> &a, const vector<ll> &b) { // 整数卷积；空输入返回空
        if (a.empty() || b.empty()) return {};
        int z = a.size() + b.size() - 1, n = 1;
        while (n < z) n <<= 1;
        vector<C> f(n), g(n);
        for (int i = 0; i < (int)a.size(); ++i) f[i].x = a[i];
        for (int i = 0; i < (int)b.size(); ++i) f[i].y = b[i];
        fft(f, false);
        for (int i = 0; i < n; ++i) {
            int j = (n - i) & (n - 1);
            C A = (f[i] + f[j].conj()) * C(0.5, 0);
            C B = (f[i] - f[j].conj()) * C(0, -0.5);
            g[i] = A * B;
        }
        fft(g, true);
        vector<ll> c(z);
        for (int i = 0; i < z; ++i) c[i] = llround(g[i].x);
        return c;
    }

    vector<double> fft_mul(const vector<double> &a, const vector<double> &b) { // 浮点卷积
        if (a.empty() || b.empty()) return {};
        int z = a.size() + b.size() - 1, n = 1;
        while (n < z) n <<= 1;
        vector<C> f(n), g(n);
        for (int i = 0; i < (int)a.size(); ++i) f[i].x = a[i];
        for (int i = 0; i < (int)b.size(); ++i) f[i].y = b[i];
        fft(f, false);
        for (int i = 0; i < n; ++i) {
            int j = (n - i) & (n - 1);
            C A = (f[i] + f[j].conj()) * C(0.5, 0);
            C B = (f[i] - f[j].conj()) * C(0, -0.5);
```

```
            g[i] = A * B;
        }
        fft(g, true);
        vector<double> c(z);
        for (int i = 0; i < z; ++i) c[i] = g[i].x;
        return c;
    }
```

调用与 NTT 一样：

```
vector<ll> a = {1, 2, 3}, b = {4, 5};
vector<ll> c = fft_mul(a, b); // {4,13,22,15}

vector<double> x = {1.5, 2.0}, y = {1.0, 0.5};
vector<double> z = fft_mul(x, y); // 浮点卷积，不取整
```

理论复杂度为 $O(n \log n)$。FFT 使用 `double`，正确取整的充分条件是每个结果系数的累计数值误差严格小于 `0.5`，不能只看最终答案是否放得进 `ll`。常见的"长度约 2×10^5、输入绝对值约 10^4"通常较稳，但这不是数学保证；遇到大系数、强卡精度或需要取模时应改用 NTT/CRT。

输入可以为负数，`llround` 会按最近整数正确处理正负数；不要写 `x+0.5` 强转，否则负数会错。若卷积真实结果超出 `ll`，返回类型本身就会溢出，FFT 无法解决这个问题。

需要更高浮点精度时，可以把 `C` 的成员、`PI` 和三角函数参数统一改成 `long double`，并使用 `acosl/cosl/sinl`；代价是速度和内存开销增加。MSVC 下 `long double` 通常仍等同于 `double`，GNU x86-64 下通常是 80 位扩展精度，具体以评测环境为准。

FFT 常见用途

- 普通整数多项式乘法：直接调用 `fft_mul`。
- 大整数乘法：把十进制串从低位起每 3 或 4 位切成一个 `ll`，卷积后从低位向高位进位。
- 字符串匹配：将字符是否出现编码成 0/1 数组，把模式串反转后卷积；某个位置的卷积值就是匹配计数。
- 差值/和的计数：把值域频次数组作卷积，再读取对应下标。

大整数建议使用基数 `1000` 或 `10000`。基数越大，数组越短但浮点误差越大；极长数字优先减小基数。

---

5. 任意模数卷积：三模 NTT + CRT

当目标模数不是 NTT 友好模数时，分别在三个 NTT 质数下卷积，再用 CRT 合并。它没有 FFT 舍入误差；只要真实卷积系数小于三个质数的乘积，重构就是完全精确的。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using i128 = __int128_t;
using poly = vector<ll>;
```

```
    ll pw(ll a, ll b, ll p) {
        ll r = 1;
        for (; b; b >>= 1, a = a * a % p) if (b & 1) r = r * a % p;
        return r;
    }

    template<ll P, ll G>
    void ntt(poly &a, bool iv) { // a.size() 为 2 的幂；iv=false 正变换
        int n = a.size();
        for (int i = 1, j = 0; i < n; ++i) {
            int b = n >> 1;
            for (; j & b; b >>= 1) j ^= b;
            j ^= b; if (i < j) swap(a[i], a[j]);
        }
        for (int m = 2; m <= n; m <<= 1) {
            ll wm = pw(G, (P - 1) / m, P);
            if (iv) wm = pw(wm, P - 2, P);
            for (int p = 0; p < n; p += m) {
                ll w = 1;
                for (int i = 0; i < (m >> 1); ++i, w = w * wm % P) {
                    ll x = a[p + i], y = a[p + i + (m >> 1)] * w % P;
                    a[p + i] = x + y < P ? x + y : x + y - P;
                    a[p + i + (m >> 1)] = x - y >= 0 ? x - y : x - y + P;
                }
            }
        }
        if (iv) {
            ll x = pw(n, P - 2, P);
            for (ll &v : a) v = v * x % P;
        }
    }

    template<ll P, ll G>
    poly conv(const poly &a, const poly &b) { // 返回模 P 卷积
        if (a.empty() || b.empty()) return {};
        int z = a.size() + b.size() - 1, n = 1;
        while (n < z) n <<= 1;
        poly x(n), y(n);
        for (int i = 0; i < (int)a.size(); ++i) x[i] = (a[i] % P + P) % P;
        for (int i = 0; i < (int)b.size(); ++i) y[i] = (b[i] % P + P) % P;
        ntt<P, G>(x, false); ntt<P, G>(y, false);
        for (int i = 0; i < n; ++i) x[i] = x[i] * y[i] % P;
        ntt<P, G>(x, true); x.resize(z); return x;
    }

    poly any_mul(poly a, poly b, ll mod) { // 返回 a*b mod mod；要求 mod>=1，见代码外系
数界
        assert(mod >= 1);
        if (a.empty() || b.empty()) return {};
        if (mod == 1) return poly(a.size() + b.size() - 1);
        for (ll &x : a) x = (x % mod + mod) % mod;
```

```
        for (ll &x : b) x = (x % mod + mod) % mod;
        int z = a.size() + b.size() - 1, n = 1;
        while (n < z) n <<= 1;
        assert(n <= (1 << 21));

        const ll p1 = 998244353, p2 = 1004535809, p3 = 469762049;
        poly a1 = conv<p1, 3>(a, b), a2 = conv<p2, 3>(a, b), a3 = conv<p3, 3>(a, b);
        static const ll i12 = pw(p1 % p2, p2 - 2, p2);
        static const ll p12_3 = (ll)((i128)p1 * p2 % p3);
        static const ll i123 = pw(p12_3, p3 - 2, p3);
        poly c(a1.size());
        for (int i = 0; i < (int)c.size(); ++i) {
            ll t1 = (a2[i] - a1[i]) % p2;
            if (t1 < 0) t1 += p2;
            t1 = t1 * i12 % p2;
            ll x12 = a1[i] + p1 * t1;
            ll t2 = (a3[i] - x12 % p3) % p3;
            if (t2 < 0) t2 += p3;
            t2 = t2 * i123 % p3;
            c[i] = (ll)(((i128)x12 + (i128)p1 * p2 * t2) % mod);
        }
        return c;
    }
```

调用方式：

```
ll p = 1000000007;
vector<ll> c = any_mul(a, b, p); // c=(a*b) mod p
```

三个模数的乘积约为 4.71×10^{26}。代码先把输入规范到 `[0,p)`；要保证无歧义，需满足每个真实卷积系数

$$\sum_j a_j b_{k-j} < 998244353 \times 1004535809 \times 469762049.$$

一个容易检查的充分条件是

$$\min(|A|, |B|)(p-1)^2 < 4.71 \times 10^{26}.$$

例如 `p≈1e9`、较短多项式长度不超过 `1e5` 时满足该界。若目标模数恰好是三个 NTT 模数之一，直接使用对应单模 NTT 更快，也不需要上述系数界。

由于 `1004535809-1` 只含 2^{21}，本模板支持的最大变换长度是 2^{21}；也就是 `a.size()+b.size()-1 <= 2^21`。一次任意模卷积共做 9 次 NTT，常数显著高于单模 NTT，但结果可证明精确。

---

6. 高频题型如何转成卷积

6.1 多项式乘法

若

$$A(x)=\sum_{i=0}^{n}a_ix^i,\qquad B(x)=\sum_{j=0}^{m}b_jx^j,$$

则 `mul(a,b)[k]` 等于 $\sum_{i+j=k}a_ib_j$。输入数组长度分别为 `n+1`、`m+1`，输出长度为 `n+m+1`。

6.2 下标和计数

若 `a[i]`、`b[j]` 分别表示值 `i`、`j` 的出现次数，则卷积 `c[s]` 是有序对满足 `i+j=s` 的数量。计数可能很大，不取模时确认结果能放进 `ll`。

6.3 下标差计数

要统计 `i-j=d`，把第二个数组反转。若 `b` 的最大下标为 `M`，令 `rb[M-j]=b[j]`，则 `mul(a,rb)[M+d]` 就是差为 `d` 的配对数。

6.4 字符串匹配

文本指示数组为 `a`，模式指示数组反转后为 `b`。长度为 `m` 的模式从文本位置 `s` 开始时，对应卷积下标 `s+m-1`。多字符完全匹配可对每种字符分别卷积并累加，或使用平方差编码：

$$\sum_i(T_{s+i}-P_i)^2=\sum_i T_{s+i}^2-2\sum_i T_{s+i}P_i+\sum_i P_i^2.$$

中间只有交叉项需要卷积。存在通配符时，让通配位置贡献 0，或使用专门的三次方匹配恒等式。

6.5 线性递推与生成函数

卷积是形式幂级数运算的基础。常见关系包括：

- $B=A^{-1}$：递推卷积方程 $AB=1$；
- $B=\ln A$：$B'=A'/A$；
- $B=\exp A$：$\ln B=A$；
- 分治优化 DP：把已知左半段对右半段的转移写成一次卷积。

遇到“转移只与下标差有关”的 DP，应优先检查能否做 CDQ 分治卷积或生成函数。

---

7. 边界、复杂度与易错点

1. `vector` 下标就是次数。长度 `n` 表示保留 $x^0\sim x^{n-1}$，不是最高次数为 `n`。
2. NTT 长度必须是 2 的幂。主板子的 `mul` 会自动补零；直接调用 `ntt` 时由调用者保证。
3. `998244353=119\times2^{23}+1`，所以主板子最大变换长度为 2^{23}。
4. `2^23` 是变换数组长度，不是任意 FPS 的可保留项数。当前短板子的截断乘法仍先计算完整卷积，两个接近 `n` 项的多项式通常需要约 `2n` 的变换长度，因此通用 FPS 目标长度一般不要超过 2^{22}；更长需要专门的半卷积/中间乘积优化。
5. `inv` 要求常数项非零；`ln` 要求常数项为 1；`exp` 要求常数项为 0；`compose/revert` 要求内层多项式常数项为 0。

6. 零多项式在 `trim/divmod` 中用空 `vector` 表示，但 FPS 接口按要求返回固定 `n` 项时会返回 `vector<ll>(n)`。
7. 输入有负系数时必须先规范取模。主板子的公开运算已经处理，若自己直接调用 `ntt`，需先把每项放到 `[0,mod)`。
8. 只需前 `n` 项时要尽早 `resize(n)`，否则中间卷积可能多做一倍工作并占用更多内存。
9. FFT 的"最终值不超过 `ll`"不等于"必然精确"；精度取决于长度、系数、算法误差和平台实现。
10. 三模 CRT 的任意模卷积按 `[0,p)` 代表元重构非负整数系数，必须检查第 5 节的系数界。若要重构有符号整数，应在 CRT 后以三个质数乘积的一半为界选取中心代表元。
11. 多项式插值点必须互异；多点求值则允许重复点，只是没有必要重复计算。
12. GCC/Clang 下 `__int128` 可用但不能直接 `cin/cout`；三模 CRT 只把它用于中间乘法。

典型复杂度：

操作	时间复杂度
一次 NTT / FFT	$O(n \log n)$
卷积	$O(n \log n)$
FPS 逆、ln、exp、sqrt、pow	$O(n \log n)$
Bostan-Mori / 线性递推第 k 项	$O(M(n) \log k)$
Berlekamp-Massey（给出 m 项）	$O(m^2)$
多项式除法、泰勒平移	$O(n \log n)$
多项式 gcd / 模多项式快速幂	$O(M(n) \log n)$ / $O(M(n) \log n \log k)$
多点求值、插值	$O(n \log^2 n)$
连续点单次拉格朗日求值	$O(n)$
多项式复合、复合逆（本模板）	$O(\sqrt{n}\, M(n))$
CDQ 卷积递推	$O(n \log^2 n)$
XOR / OR / AND 卷积	$O(n \log n)$
子集卷积（全集大小 k）	$O(k^2 2^k)$

---

8. 范围取舍：哪些没有塞进主板子

到这里已经覆盖绝大多数 XCPC 常见的 NTT、FFT、FPS、线性递推和位运算卷积题。以下内容不是忘记，而是因为使用频率低、代码长或前置条件复杂，建议遇到明确题型时单独准备：

- **Bluestein / Chirp-Z 任意长度 DFT**：用于变换长度不是 2 的幂或只求等比点值。NTT 下还要检查所需单位根是否存在，普通卷积不能无条件替代。

- **任意模数 FPS 全家桶**：卷积可用第 5 节三模 NTT，但积分、ln、exp 还要求相应整数在目标模数下可逆；合数模下往往没有统一定义。不能只把 `mod` 改掉。
- **拆系数 FFT 的任意模卷积**：通常比三模 NTT 快，但存在浮点精度边界。这里优先保留可证明正确的 CRT 版本。
- **超快多项式复合/复合逆**：存在更低复杂度算法，但实现和常数都很大；第 1.6 节版本只面向中等长度。
- **松弛卷积（relaxed convolution）**：适合真正在线、逐项产生系数的场景。一般离线题可用第 3.2 节 CDQ 或生成函数解决。
- **多项式因式分解、求根、有限域不可约性算法**：会用到 `pgcd/powmod`，但属于有限域多项式专题，不是 NTT/FFT 基础板子的合理范围。
- **Min-Plus / Max-Plus 卷积**：它们不在普通环卷积中，不能直接使用 NTT/FFT；通常需要凸性、单调性或其他题目结构。

因此，若题目属于常规模多项式、整数卷积、生成函数、线性递推、卷积 DP、多点求值或集合卷积，这份板子已经具备完整主干；若题面明确出现上述关键词，再补对应专题模板更稳。